

Homework solution review: agentic loop

- Can you run a real generate-test-fix loop?

- Can you run a real generate-test-fix loop?
- Can you keep tests and API contracts stable?

- Can you run a real generate-test-fix loop?
- Can you keep tests and API contracts stable?
- Can you evaluate model output?

Solution folder map

```
week08homework/  
├── agent_loop.py  
├── agent_loop_output.txt  
├── bayes_factor.py  
├── gemini_simple_api.py  
├── reflection.md  
├── task.txt  
└── tests/  
    └── test_bayes_factor.py
```

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.
- Provides a clear and concise API contract: class name, constructor signature, and method signatures.

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.
- Provides a clear and concise API contract: class name, constructor signature, and method signatures.
- Provides input validation contracts for the constructor and likelihood method.

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.
- Provides a clear and concise API contract: class name, constructor signature, and method signatures.
- Provides input validation contracts for the constructor and likelihood method.
- Adds guardrails/constraints (e.g., do not modify tests, do not change the constructor signature, etc.).

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.
- Provides a clear and concise API contract: class name, constructor signature, and method signatures.
- Provides input validation contracts for the constructor and likelihood method.
- Adds guardrails/constraints (e.g., do not modify tests, do not change the constructor signature, etc.).

Example good prompt

- Provides details about the task: Mathematical requirements, Prior specifications, Implementation constraints.
- Provides a clear and concise API contract: class name, constructor signature, and method signatures.
- Provides input validation contracts for the constructor and likelihood method.
- Adds guardrails/constraints (e.g., do not modify tests, do not change the constructor signature, etc.).

 My prompt

```
BayesFactor(n, k)  
likelihood(theta)  
evidence_slab(), evidence_spike()  
bayes_factor()
```

slab: $U(0, 1)$

spike: $U(0.47, 0.53)$

slab: $U(0, 1)$

spike: $U(0.47, 0.53)$

- Not a point mass at 0.5 – that's something the LLM would guess if you didn't specify it.

$$\text{evidence} = \int \text{likelihood}(\theta) * \text{prior}(\theta) d\theta$$

$$\text{evidence} = \int \text{likelihood}(\theta) * \text{prior}(\theta) d\theta$$

- Could compute slab and spike evidence separately.

$$\text{evidence} = \int \text{likelihood}(\theta) * \text{prior}(\theta) d\theta$$

- Could compute slab and spike evidence separately.
- Could also define a single integral function with arbitrary bounds.

Many submissions resulted in errors

- Some prompts never stated the explicitly which evidence to put in the numerator.

Many submissions resulted in errors

- Some prompts never stated the explicitly which evidence to put in the numerator.
- Some prompts did not specify the prior density or specified it in a difficult to parse way (e.g., “see Etz et al., 2018”).

Many submissions resulted in errors

- Some prompts never stated the explicitly which evidence to put in the numerator.
- Some prompts did not specify the prior density or specified it in a difficult to parse way (e.g., “see Etz et al., 2018”).
- Some prompts contradicted the tests.

Many submissions resulted in errors

- Some prompts never stated the explicitly which evidence to put in the numerator.
- Some prompts did not specify the prior density or specified it in a difficult to parse way (e.g., “see Etz et al., 2018”).
- Some prompts contradicted the tests.
- Result: loops get stuck in whack-a-mole mode.

```
self.assertAlmostEqual(  
    self.bf_chance.bayes_factor(), 3/2, places=2  
)
```

```
self.assertAlmostEqual(  
    self.bf_chance.bayes_factor(), 3/2, places=2  
)
```

- Multiple numeric anchors constrain ambiguous implementations.

```
self.assertAlmostEqual(  
    self.bf_chance.bayes_factor(), 3/2, places=2  
)
```

- Multiple numeric anchors constrain ambiguous implementations.
- Single numeric anchor can invite **coding to the test**.

What passing tests should prove

- API shape matches expected calls.

What passing tests should prove

- API shape matches expected calls.
- Core math behavior matches anchor cases.

What passing tests should prove

- API shape matches expected calls.
- Core math behavior matches anchor cases.
- Required exceptions and messages are correct.

What passing tests does not prove

- Numerical stability for all scenarios.

What passing tests does not prove

- Numerical stability for all scenarios.
- Good naming, comments, and maintainability.

What passing tests does not prove

- Numerical stability for all scenarios.
- Good naming, comments, and maintainability.
- Resistance to prompt/test contradictions.

```
for attempt in range(1, MAX_ATTEMPTS + 1):  
    files, notes = client.prompt(...)  
    code, output = run_tests()
```

```
for attempt in range(1, MAX_ATTEMPTS + 1):  
    files, notes = client.prompt(...)  
    code, output = run_tests()
```

- Retry and feedback are first-class features.


```
for attempt in range(1, MAX_ATTEMPTS + 1):  
    files, notes = client.prompt(...)  
    code, output = run_tests()
```

- Retry and feedback are first-class features.
- Passing the test suite makes the task a lot easier for the LLM.

```
for attempt in range(1, MAX_ATTEMPTS + 1):  
    files, notes = client.prompt(...)  
    code, output = run_tests()
```

- Retry and feedback are first-class features.
- Passing the test suite makes the task a lot easier for the LLM.
- Passing the context of a previous turn also helps.

```
protected_directories=[TEST_DIR]
```

```
protected_directories=[TEST_DIR]
```

- The model will still try to write to `tests/` even if you tell it not to!

```
protected_directories=[TEST_DIR]
```

- The model will still try to write to `tests/` even if you tell it not to!
- Here the API layer blocks writes into `tests/`.

```
protected_directories=[TEST_DIR]
```

- The model will still try to write to `tests/` even if you tell it not to!
- Here the API layer blocks writes into `tests/`.
- You can also restore the test file after each attempt (but this may confuse the LLM).

- API/setup failures.

- API/setup failures.
- Retries with delay recovered transient API errors (e.g., 500 errors, not your fault).

- API/setup failures.
- Retries with delay recovered transient API errors (e.g., 500 errors, not your fault).
- Prompt/test contradictions will block convergence (e.g., `BayesFactor(n,k,a,b)` vs `BayesFactor(n,k)`).

- API/setup failures.
- Retries with delay recovered transient API errors (e.g., 500 errors, not your fault).
- Prompt/test contradictions will block convergence (e.g., `BayesFactor(n,k,a,b)` vs `BayesFactor(n,k)`).
- Agree on contract before running the loop.